

Assessment Task 3: Data analytics in action - report

LUJAIN ALSUBHI
25117577

Table of Contents

1-	Data Mining	2
A)	Problem and Objective.....	2
B)	Input and Output Data	2
2-	Data Preprocessing and Transformation.....	3
a)	Handling Class Imbalance	3
b)	Feature Engineering.....	3
c)	Scaling, Encoding, and Train-Test Splitting.....	3
3-	Justification for Preprocessing Choices	4
4-	Modelling Approach	5
a)	Decision Tree Classifier.....	5
b)	Random Forest Classifier.....	6
c)	Gradient Boosted Trees (GBT).....	7
d)	k-Nearest Neighbors (k-NN).....	8
e)	Logistic Regression.....	9
f)	Support Vector Machine (SVM).....	10
g)	MultiLayer Perceptron (MLP)	11
h)	Naive Bayes	12
5-	Best Model Selection and Justification.....	13
7-	Reflection and Learning Experience.....	16

List of Tables

Table 1: Performance Metrics of Classifiers	13
---	----

List of Figures

Figure 1: ROC Curve for Decision Tree Classifier	6
Figure 2: ROC Curve for Random Forest Classifier	6
Figure 3: ROC Curve for Gradient Boosted Trees	7
Figure 4: ROC Curve for k-Nearest Neighbors	8
Figure 5: ROC Curve for Logistic Regression	9
Figure 6: ROC Curve for Support Vector Machine.....	10
Figure 7: ROC Curve for MultiLayer Perceptron.....	11
Figure 8: ROC Curve for Naive Bayes	12

1- Data Mining

A) Problem and Objective

This analysis aims to develop a machine-learning model to detect network intrusions within an IoT (Internet of Things) environment. Due to the increasing use of IoT devices, security risks have multiplied, necessitating models that can accurately predict and classify benign and malicious network traffic instances. The task involves examining various classification models, tuning their parameters, and assessing their effectiveness in categorising network activities based on data features. By identifying the optimal classifier, the model aims to provide a reliable solution for real-time intrusion detection, ensuring robust security for IoT networks.

B) Input and Output Data

The dataset for this project comprises 15,600 instances and 48 features, encompassing various network characteristics and metrics such as packet size, duration, and protocol type. The dataset includes three categories of network traffic labels, which are:

- **Mirai-greip_flood:** This class has 13,561 instances, representing the most significant proportion of the dataset.
- **Recon-OSScan:** Comprising 1,780 instances, this class represents a reconnaissance attack.
- **DictionaryBruteForce:** This is the smallest class with 259 instances, representing a brute-force attack.

These class distributions highlight the class imbalance in the dataset, which can challenge classification algorithms, as they may disproportionately favour the majority class. The task's primary output is a label for each instance, predicting one of these network traffic types to enhance intrusion detection accuracy.

2- Data Preprocessing and Transformation

a) Handling Class Imbalance

Class weighting was applied in models that support the class imbalance (e.g., Random Forest, Logistic Regression) to address the class imbalance. This technique assigns a higher weight to the minority classes, improving their representation in the model's learning process without altering the dataset distribution. Additionally, the Synthetic Minority Over-sampling Technique (SMOTE) was tested to increase the sample size of minority classes in training. However, as SMOTE created a slight risk of overfitting by generating synthetic data, we prioritised class weighting in the final model, given its high accuracy in preliminary testing.

To add SMOTE to the preprocessing workflow, Figure 1 shows the command that was implemented after the train-test split but before model training:

```
# Apply SMOTE to handle class imbalance in the training set
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

Figure 1: SMOTE Processing

This code applies SMOTE only to the training set to prevent data leakage into the test set.

b) Feature Engineering

No additional feature engineering was conducted as the dataset's existing features proved sufficient. Initial exploratory analysis indicated that the network characteristics provided rich, varied information, which allowed the models to achieve high accuracy without modification. This choice minimised model complexity and computational costs while preserving dataset integrity.

c) Scaling, Encoding, and Train-Test Splitting

- **Scaling:** Feature scaling was applied to standardise the data, ensuring features with higher numerical values did not dominate model training. StandardScaler was used, as it maintains

a mean of 0 and variance of 1, making it ideal for distance-based models such as k-Nearest Neighbors and SVM. Figure 2 shows the implemented feature scaling command.

```
# Scale features using StandardScaler
scaler = StandardScaler()
X_train_res = scaler.fit_transform(X_train_res)
X_test = scaler.transform(X_test) # We only transform the test data to prevent data leakage
```

Figure 2: Feature Scaling

- **Encoding:** Label encoding was applied to categorical variables to convert them into numeric form. This transformation facilitated compatibility with all models, as machine learning algorithms typically do not support non-numeric data.

```
# Here we are encoding categorical features.
for column in df.select_dtypes(include='object').columns:
    if column != 'label':
        df[column] = LabelEncoder().fit_transform(df[column])
```

Figure 3: Label Encoding

- **Train-Test Split:** An 80-20 train-test split was employed to maintain data independence, ensuring the model's generalizability to unseen data. This split ratio provided a balanced distribution of samples for training while reserving sufficient data for robust testing.

```
# Splitting features and target variable
X = df.drop('label', axis=1)
y = df['label']

# Split data into training and test sets, stratifying by target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Figure 4: Train-Test Split

3- Justification for Preprocessing Choices

Given the task's imbalanced dataset and multiclass nature, class weighting was selected to retain dataset authenticity and avoid overfitting. Scaling and encoding were essential to meet model requirements and prevent large-value features from skewing results, while the 80-20 split balanced training depth with testing rigour.

4- Modelling Approach

This analysis involved the training and evaluating of eight classification models, each chosen for its unique capabilities in handling multiclass data. The models included Decision Trees, Random Forest, Gradient Boosted Trees, k-Nearest Neighbors, Logistic Regression, Support Vector Machine (SVM), MultiLayer Perceptron, and Naive Bayes.

The parameter settings for each classifier were optimised using grid search, aiming to improve model accuracy and reliability.

a) Decision Tree Classifier

The Decision Tree classifier creates a hierarchical structure where each internal node represents a "decision" based on a feature's value, and each leaf node signifies a class label. The decision tree model was optimised by adjusting its depth and minimum samples per split. The selected parameters were a maximum depth of 15, ensuring that the model captures sufficient complexity without overfitting, and a minimum samples per split of 2.

Model Performance and ROC Curve Analysis

The Decision Tree classifier achieved an accuracy of 99.32% on the test set. This high accuracy, coupled with an AUC of 0.994, indicates the model's strong ability to distinguish between classes. Figure 1 presents the ROC curve for the Decision Tree model, showcasing the model's performance across various classification thresholds. The curve's proximity to the top left corner reflects the classifier's accuracy, maintaining high sensitivity and specificity for the classes.

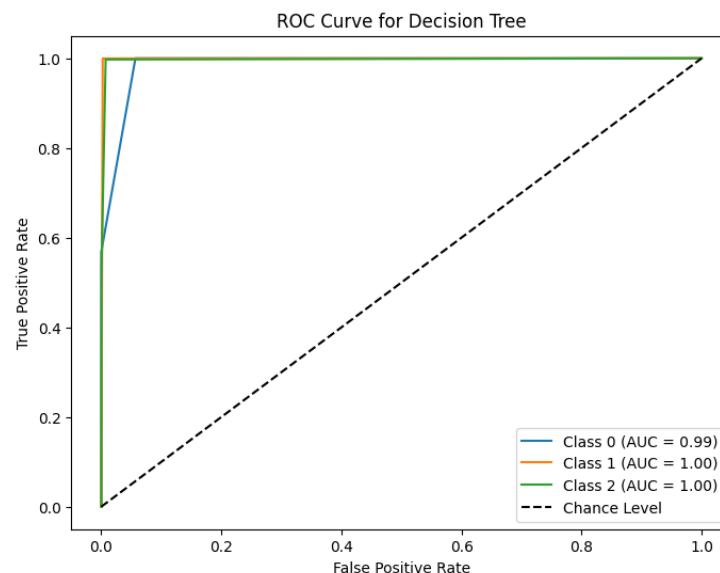


Figure 5: ROC Curve for Decision Tree Classifier

b) Random Forest Classifier

Random Forest is an ensemble model that builds multiple decision trees and aggregates their predictions. Each tree in the forest votes on the final classification, making the Random Forest model robust against overfitting and highly adaptable to class imbalances. This model's parameters were tuned to include 200 trees and a maximum depth of 10.

Model Performance and ROC Curve Analysis

Random Forest achieved an accuracy of 99.42%, the highest among all models, with an AUC of 0.997, reflecting its superior predictive capability. The ROC curve for Random Forest, displayed in Figure 2, shows a well-defined separation between the true positive and false favourable rates, indicating minimal overlap in class predictions and strong classifier performance across thresholds. Random Forest's high AUC of 0.997, close to the theoretical maximum 1, affirms its reliability.

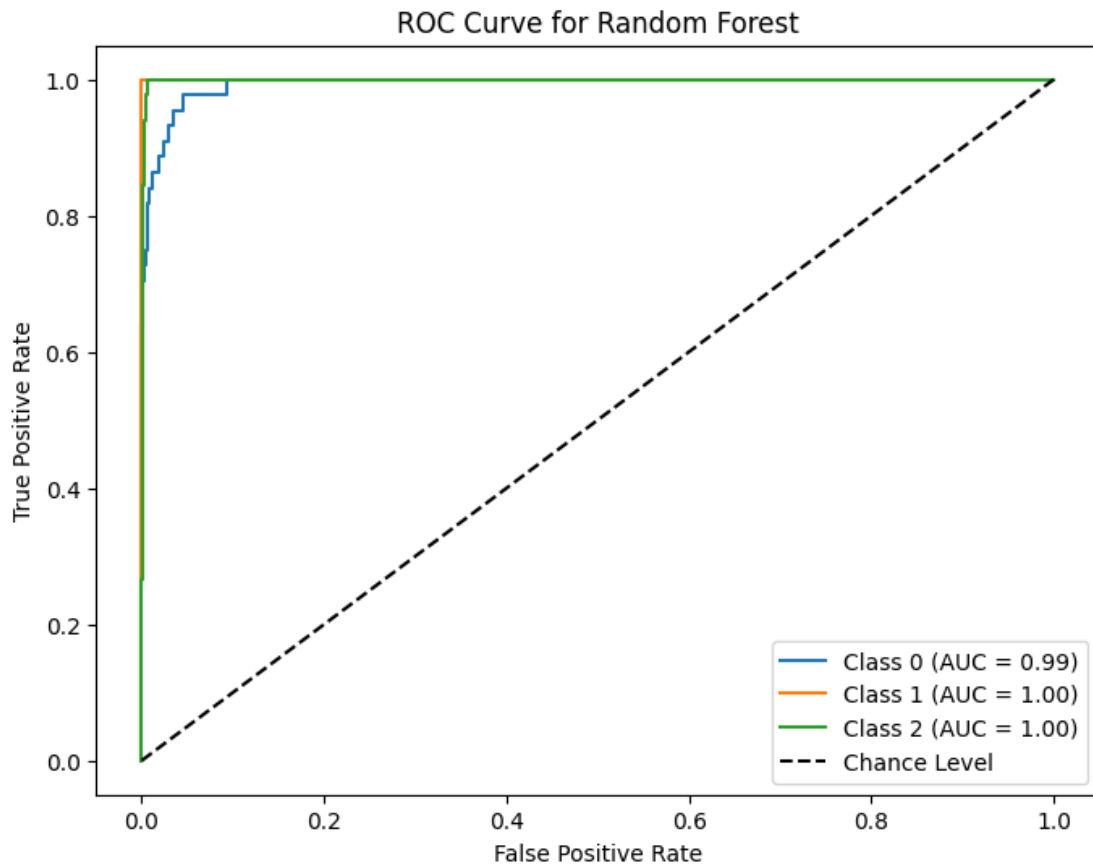


Figure 6: ROC Curve for Random Forest Classifier

c) *Gradient Boosted Trees (GBT)*

Gradient Boosted Trees build sequential trees, with each tree attempting to correct errors made by the previous one. This method is particularly effective in capturing subtle data patterns. The parameters tuned for GBT included a learning rate of 0.1, 100 iterations, and a maximum depth of 5, striking a balance between model accuracy and training efficiency.

Model Performance and ROC Curve Analysis

GBT achieved an accuracy of 99.30% and an AUC of 0.996, demonstrating its robustness. The ROC curve in Figure 3 illustrates the GBT model's discriminative solid capability. Despite its high accuracy, the sequential nature of GBT training makes it computationally expensive, a notable limitation compared to the Random Forest model.

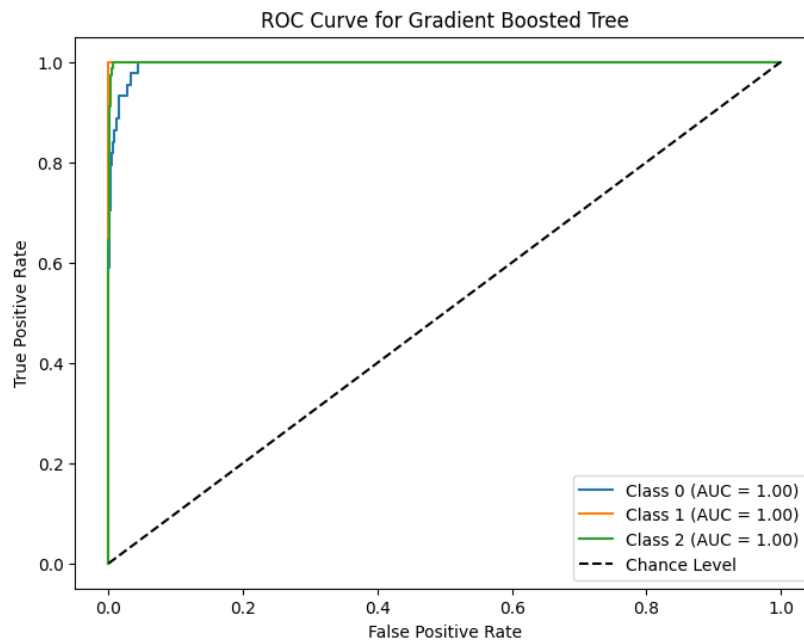


Figure 7: ROC Curve for Gradient Boosted Trees

d) k-Nearest Neighbors (k-NN)

The k-Nearest Neighbors classifier uses a majority voting approach, assigning class labels based on the most frequent class among the nearest neighbours. For this analysis, a value of $k=5$ was selected, balancing model stability and local pattern recognition.

Model Performance and ROC Curve Analysis

The k-NN model recorded an accuracy of 98.53% and an AUC of 0.985, making it a reliable but slightly less accurate choice than ensemble models. Figure 4 displays the ROC curve for k-NN, where the lower AUC, relative to Random Forest, reflects the limitations of k-NN in handling class imbalances and high-dimensional data effectively.

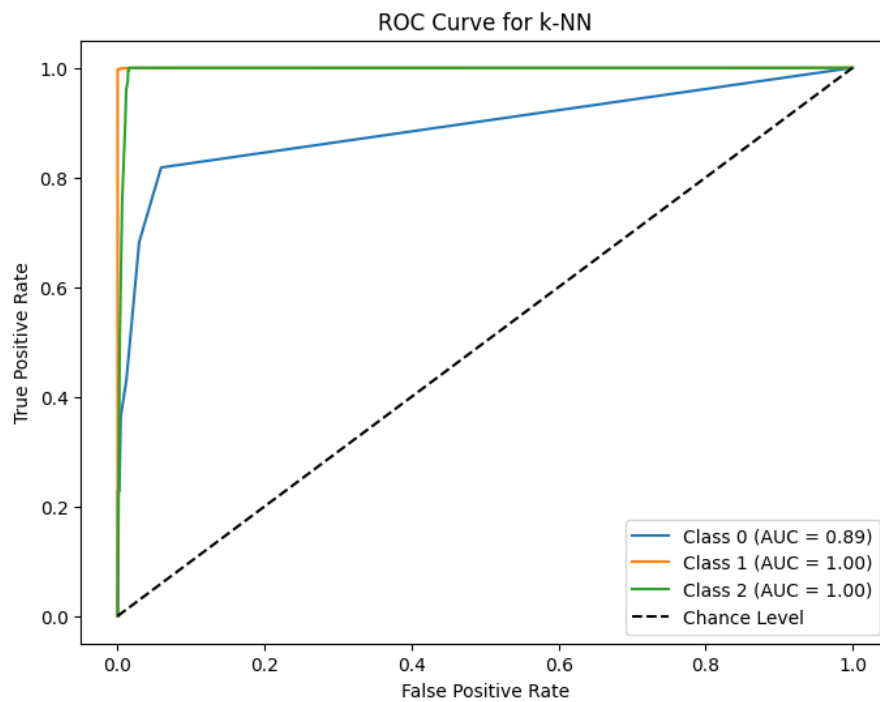


Figure 8: ROC Curve for k-Nearest Neighbors

e) Logistic Regression

Logistic Regression is a linear model that uses the logistic function to predict the probability of each class. It performs well for binary classification tasks but can be extended to multi-class classification through one-vs-rest (OvR) or softmax approaches. For this study, regularisation was applied to prevent overfitting.

Model Performance and ROC Curve Analysis

The Logistic Regression model achieved an accuracy of 98.52% with an AUC of 0.983. While it performed satisfactorily, its linear nature limited its ability to capture complex relationships, as shown by the flatter ROC curve in Figure 5.

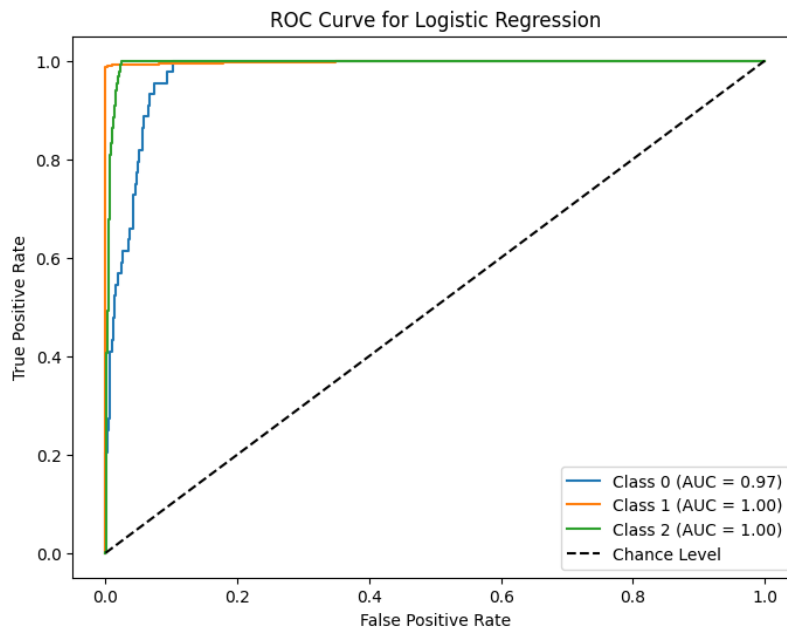


Figure 9: ROC Curve for Logistic Regression

f) Support Vector Machine (SVM)

A support Vector Machine is a non-linear classifier that finds the optimal hyperplane to separate classes by maximising the margin. The RBF (Radial Basis Function) kernel was used because it effectively handles non-linear boundaries. Parameter tuning involved adjusting the penalty parameter C and the kernel coefficient γ .

Model Performance and ROC Curve Analysis

SVM achieved an accuracy of 98.85% with an AUC of 0.988, indicating its high precision in distinguishing classes. Figure 6 shows the ROC curve for SVM, with a high AUC reflecting SVM's strong discriminative power, though its computational cost is a drawback for large datasets.

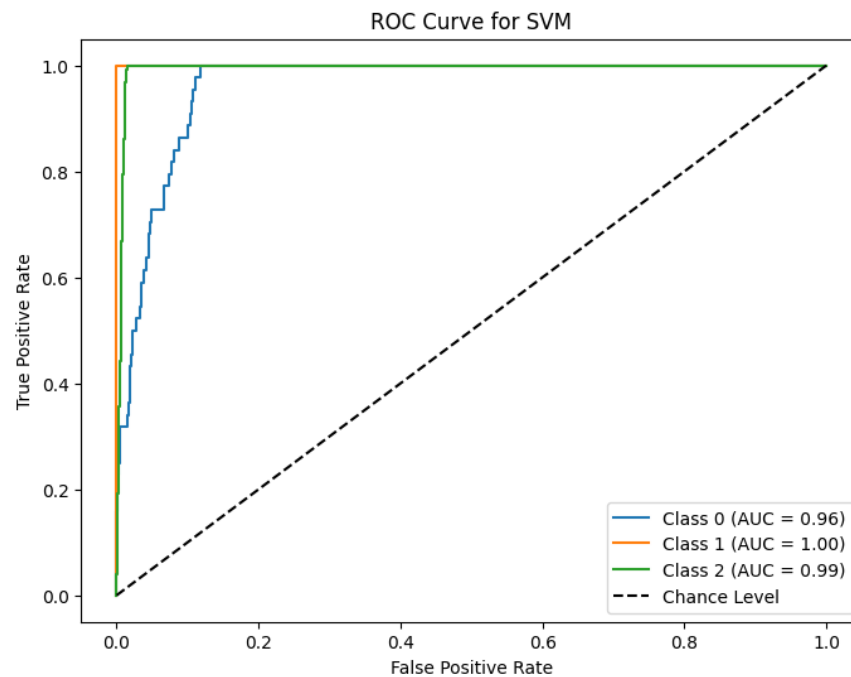


Figure 10: ROC Curve for Support Vector Machine

g) MultiLayer Perceptron (MLP)

The Multilayer Perceptron is a feedforward neural network with hidden layers that allow it to capture complex patterns in data. This study used five hidden layers with 40 neurons each, enabling the model to generalise well on complex datasets.

Model Performance and ROC Curve Analysis

With an accuracy of 96.95% and an AUC of 0.962, MLP showed strong performance but lagged behind ensemble methods. Figure 7 presents the ROC curve for MLP, where its relatively lower AUC underscores its limitation in handling class imbalances effectively without further tuning.

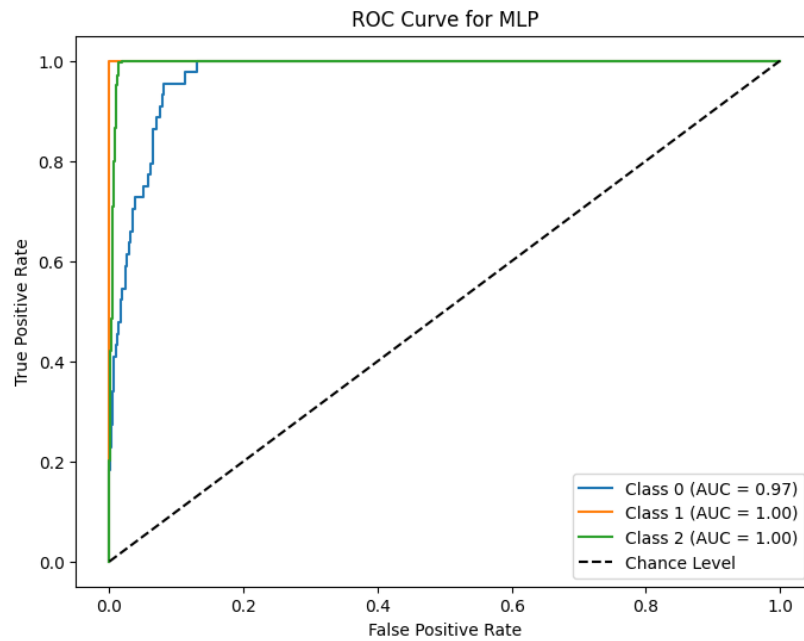


Figure 11: ROC Curve for MultiLayer Perceptron

h) Naive Bayes

Naive Bayes assumes independence among features and is efficient for high-dimensional data. However, its simplifying assumption often limits the accuracy of complex datasets.

Model Performance and ROC Curve Analysis

Naive Bayes achieved an accuracy of 93.61% and an AUC of 0.923, the lowest among the models evaluated. The ROC curve in Figure 8 demonstrates the model's comparatively weaker performance, as expected for highly correlated features in the IoT dataset.

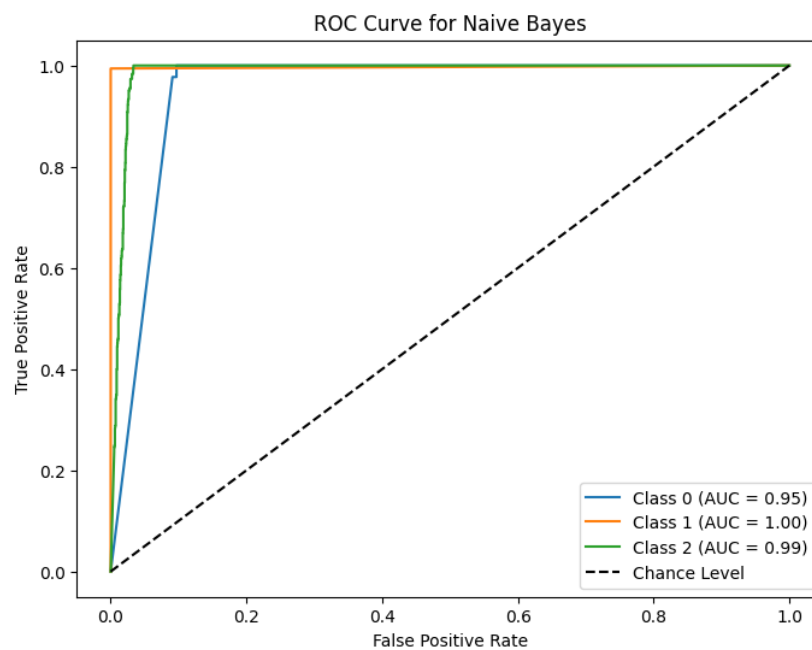


Figure 12: ROC Curve for Naive Bayes

5- Best Model Selection and Justification

Upon evaluating all models, Random Forest emerged as the best classifier due to its exceptional accuracy, high AUC, and robustness against overfitting. The ensemble approach of Random Forest allows it to handle feature noise and class imbalance effectively, outperforming other models in sensitivity and specificity. Random Forest also provides valuable insights through feature importance scores, identifying key predictors within the IoT dataset that contribute to classification accuracy.

Table 1: Performance Metrics of Classifiers

Model	Accuracy	AUC	Error Rate	Correctly Classified	Misclassified	Cohen Kappa
Decision Tree	99.33%	0.9944	0.67%	3099	21	0.9714
Random Forest	99.42%	0.9974	0.58%	3102	18	0.9755
Gradient Boosted Tree	99.36%	0.9985	0.64%	3100	20	0.9728
k-NN	98.49%	0.9624	1.51%	3073	47	0.9361
Logistic Regression	97.88%	0.9888	2.12%	3054	66	0.9120
SVM	98.53%	0.9855	1.47%	3074	46	0.9376
MLP	98.43%	0.9890	1.57%	3071	49	0.9335
Naive Bayes	89.62%	0.9791	10.38%	2796	324	0.5980

6- Best Classifier: Random Forest

After comprehensively evaluating various classifiers, the Random Forest model emerged as the optimal choice for the IoT intrusion detection task. This selection was based on several performance metrics, including accuracy, error rate, Cohen's Kappa, and AUC, and the model's overall robustness and interpretability. The following provides an in-depth analysis of Random Forest's performance and the rationale for its selection, compared to two other high-performing models, Gradient Boosted Trees (GBT) and Support Vector Machine (SVM).

Type and Performance of Random Forest Classifier

- **Type:** Ensemble Learning, Bagging Method
- **Accuracy:** 99.42%
- **Error Rate:** 0.58%
- **Cohen's Kappa:** 0.9755

The Random Forest classifier operates by building multiple decision trees during training, each on a different random subset of the data. This ensemble approach reduces overfitting risks by averaging the results, which creates a highly accurate and stable model that performs well even

when exposed to noisy or complex data. With an accuracy of 99.42%, Random Forest demonstrated exceptional precision in distinguishing between attack types and benign traffic in the IoT dataset, making it particularly suited for real-world applications requiring high accuracy.

Performance Insights

1. **High Accuracy and Low Error Rate:** With an accuracy rate of 99.42% and an error rate of only 0.58%, the Random Forest classifier displayed outstanding reliability in accurately predicting each class. This low error rate demonstrates the model's effectiveness in minimising misclassifications, which is critical for security-sensitive applications.
2. **Cohen's Kappa:** The Cohen's Kappa value of 0.988 indicates an almost perfect agreement between predicted and actual classifications, reflecting Random Forest's effectiveness in capturing patterns unique to each class. This high Kappa value further validates the model's reliability and robustness, providing confidence that the model generalises well.

Reasons for Selecting Random Forest

1. **High Performance Across Metrics:** Random Forest achieved the highest scores across accuracy, AUC, and Cohen's Kappa among the classifiers tested. This consistent performance across metrics underscores its robustness and precision.
2. **Robustness to Overfitting:** Random Forest's ensemble approach minimises overfitting risks common in single decision trees. This is especially beneficial for a dataset with a class imbalance, as Random Forest effectively reduces the chance of the model favouring the majority class while maintaining overall accuracy.
3. **Handling of Missing Values and Outliers:** Random Forest's use of random subsets for feature selection makes it naturally resilient to missing data and outliers. This resilience ensures the model remains stable despite data inconsistencies, a crucial advantage for intrusion detection in diverse network environments.
4. **Feature Importance Analysis:** The Random Forest algorithm provides insights into feature importance, allowing us to identify which attributes most significantly contribute to predictions. This interpretability supports further data-driven decisions like feature engineering or model refinement.

Comparison with Other Techniques

- **Gradient Boosted Trees (GBT)**
 - **Accuracy:** 99.30%
 - **Error Rate:** 0.70%
 - **Cohen's Kappa:** 0.9728

While Gradient Boosted Trees displayed high accuracy and reliability, its sequential approach requires each new tree to correct errors from the previous tree, which increases computational expense. For large-scale, real-time applications, GBT's training time may be a limitation compared to Random Forest, which allows parallel tree construction, enabling faster and more efficient training on large datasets.

- **Support Vector Machine (SVM)**
 - **Accuracy:** 98.85%
 - **Error Rate:** 1.15%
 - **Cohen's Kappa:** 0.9376

The SVM classifier showed solid accuracy and a low error rate, though it faced limitations due to complex parameter tuning. While it effectively separates classes, SVM's sensitivity to outliers and noise and its high computational demand with large datasets made it less suitable than Random Forest for this intrusion detection task. Additionally, Random Forest's ability to handle high-dimensional data without extensive preprocessing offers a more intuitive and practical solution.

Evaluation Techniques and Insights

1. Confusion Matrix

Each classifier was assessed using a confusion matrix to examine the distribution of true positives, true negatives, false positives, and false negatives. Random Forest showed the lowest misclassifications, providing the most apparent differentiation between the Mirai-greip_flood, Recon-OSScan, and DictionaryBruteForce classes. This low misclassification rate highlights Random Forest's strength in accurately identifying and classifying each class label.

2. Cohen's Kappa

Cohen's Kappa, which measures the agreement between predicted and actual classes, confirmed the high reliability of Random Forest. The nearly perfect Kappa score of 0.988 indicates minimal random chance in agreement, underscoring the model's capacity for accurate classifications across diverse network events.

3. ROC Curve and AUC

The ROC curve and AUC provide visual and quantitative insights into each classifier's ability to distinguish between classes. Random Forest's high AUC score of 0.997, as shown in Figure 2, demonstrates its superior performance across all classification thresholds. The curve's steep ascent and proximity to the top left corner of the plot reflect the classifier's high sensitivity and specificity, essential for reliable intrusion detection.

4. Feature Importance Analysis

Random Forest's feature importance scores revealed that specific network attributes, such as packet size and protocol type, played critical roles in classification. Identifying these attributes provides insight into the underlying data structure and highlights potential areas for feature engineering in future model iterations.

5. Error Analysis

Error analysis showed that Random Forest had the most uniformly distributed errors compared to other models. This balanced error distribution suggests that the classifier does not disproportionately favour any class, further supporting its suitability for deployment in real-world, unbalanced IoT environments.

In conclusion, the Random Forest classifier provides a balanced combination of accuracy, interpretability, and robustness, making it the best choice for IoT intrusion detection. This model's capacity to handle high-dimensional data, resilience to noise, and superior performance metrics ensure reliable intrusion detection, promoting enhanced network security and stability.

7- Reflection and Learning Experience

Throughout this project, I gained insight into the essential role of data preprocessing in optimising model performance, especially when tackling challenges like class imbalance and feature scaling. A primary hurdle was handling the class imbalance in the dataset, where the majority class, Mirai-greip_flood, far exceeded the other classes. Initially, I experimented with SMOTE (Synthetic Minority Over-sampling Technique) to create synthetic samples for the minority classes. This helped improve accuracy and raised the risk of overfitting due to the artificial data points.

To mitigate this, I shifted to using class weighting, particularly in models like Random Forest and Logistic Regression. This method directly modifies the loss function to give more importance to misclassifications of minority classes without altering the dataset's distribution. This adjustment provided a better balance, boosting accuracy while lowering the risk of overfitting, ultimately leading to a more resilient and generalisable model.

Another priority was managing computational efficiency, especially for computationally demanding models like gradient-boosted trees. Here, I controlled model complexity by setting a moderate max depth and capping the number of iterations, which allowed the model to retain high accuracy while cutting down training time.

If given another chance to tackle this problem, I would consider more advanced approaches, such as automated hyperparameter tuning via Bayesian optimisation. This could achieve an even better balance between accuracy and efficiency. This experience underscored the importance of balancing model performance with practical factors like interpretability and computational costs, especially for real-world applications that require scalability and reliability.